

JavaScript 1.3

Functions, Closures & Functional Patterns

Print-first interview refresher | value -> call -> capture -> lifecycle -> composition

Trace four things:
function identity | invocation form
captured bindings | lifetime

1 | Function as a runtime value

The form controls creation time, invocation context and constructibility.

declaration

```
function run() {}
```

initialized before execution
own this + args | usually new

expression

```
const run = function() {}
```

created when expression runs
own this + args | usually new

named expression

```
function self() {}
```

self-name visible inside only
recursion + stack traces

arrow

```
const run = () => value
```

lexical this | no own args
not constructible | no new

binding name != function identity | assignment copies the same callable value

2 | Invocation decides regular this

```
obj.method() → const f = obj.method; f()
```

this = obj receiver lost

Explicit receiver APIs

call

```
fn.call(obj, a, b)
```

now | list

apply

```
fn.apply(obj, args)
```

now | array-like

bind

```
fn.bind(obj, a)
```

new fn | later

bound args first -> call-time args later

Repeated bind

first receiver stays; args append

Callback identity

store once for add / remove

3 | Parameters, arguments & arity

Tool	Meaning	Key rule
parameter	binding in definition	not an actual value yet
argument	value in one call	extra / missing allowed
default	initializer per call	missing or undefined
rest	remaining args -> Array	last parameter
arguments	all actual args array-like	regular functions only

Three different counts

```
function f(a, b = 2, ...rest) {}
```

f.length = 1

arguments = actual

rest = suffix

4 | Closure: function + access to live lexical bindings

reachable callback
called later

lexical environment
created where written

live binding
current value

object identity
shared / mutated

object graph
retained if reachable

Loop capture

```
for (var i...) one binding  
cb0 -> cb1 -> ... i = 3
```

[3, 3, 3]

```
for (let i...) per iteration  
cb0->i0 cb1->i1 cb2->i2
```

[0, 1, 2]

Lifecycle check

listener | timer | subscription | cache
store identity -> remove / clear / evict
Prove a retaining path; closure != leak.

Snapshot? create a separate value. Memoization? define keys + bounds + invalidation.

5 | Purity & ownership

```
input → pure fn → out
```

same inputs -> same observable result
local mutation of fresh result can be pure

Explicit effect boundary

DOM | network | storage | analytics
timers | listeners | shared mutation

Spread is shallow: track nested identity.

6 | Functional patterns

Currying

```
f(a, b, c) -> f(a)(b)(c)
```

Partial application

```
fix a -> g(b, c)
```

Composition order

```
x → g(x) → f(g(x))
```

compose right->left | pipe listed left->right

7 | Production review

- 1 Contract** values | defaults | errors
- 2 Context** invocation form | receiver
- 3 Capture** bindings | writers | identity
- 4 Lifetime** cleanup | cache bounds
- 5 Effects** ownership | I/O boundary
- 6 Clarity** names | stacks | testability

Use patterns when call sites become clearer.

Interview trace form -> parameters -> invocation -> callback -> bindings -> identity -> lifetime -> effects